CLOUD COMPOSER THEORY Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wa

1. Why should DAGs in Composer be broken into smaller workflows instead of one big monolithic DAG?

Breaking DAGs into smaller workflows is important because it makes the pipelines easier to manage, more reliable, and more maintainable. If I put everything into one big monolithic DAG:

- It becomes very complex to read and debug. If one part fails, it can be hard to isolate where the problem is.
- Small changes can require redeploying the entire DAG, which increases risk.
- The scheduler and executor may be overloaded with too many tasks in one place, slowing everything down.

By splitting into smaller DAGs, each workflow can focus on a specific business process (like data ingestion, data transformation, or report generation). This makes it easier to test, monitor, and recover from failures. If one DAG fails, it doesn't bring down the whole system.

Also, smaller DAGs allow for reusability. For example, if I have a standard ingestion DAG, multiple teams or other DAGs can trigger it without duplicating logic.

So, smaller workflows mean better reliability, scalability, and team collaboration.

2. How do you define dependencies between tasks in an Airflow DAG?

In Airflow, dependencies between tasks are defined using operators in the DAG file. The typical way is with the >> and << operators or the set_upstream() and set_downstream() methods.

For example:

task1 >> task2

means task2 will only run after task1 finishes successfully.

Another way is chaining:

task1 >> [task2, task3] >> task4

This means task2 and task3 both depend on task1, and task4 depends on both task2 and task3.

These dependencies are important because they tell Airflow the execution order, while still letting independent tasks run in parallel.

So, dependencies are simply about defining which tasks must finish before others can start, ensuring the pipeline flows correctly.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

3. What is the benefit of running tasks in parallel within a DAG?

Running tasks in parallel speeds up the workflow and makes better use of resources. Instead of waiting for one task to finish before starting the next, Airflow can execute independent tasks at the same time.

For example, if I have three data extraction tasks (pulling data from three different APIs), they don't depend on each other. If I run them sequentially, and each takes 10 minutes, the total time is 30 minutes. If I run them in parallel, all three can finish in about 10 minutes.

Parallelism also improves scalability. Airflow workers can distribute tasks across multiple machines, so big workflows complete faster.

Another benefit is resilience — if one parallel task fails, Airflow can retry it without affecting others.

So, parallel execution reduces pipeline runtime, improves efficiency, and makes workflows more responsive to SLAs.

4. Why is DAG versioning important for production data pipelines?

DAG versioning is important because production pipelines change over time, and having versions ensures stability, traceability, and safe rollbacks. If I just overwrite a DAG file with new logic, I risk breaking existing workflows without a clear way to recover.

With versioning:

- I can safely test and roll out changes in a controlled manner (for example, deploying dag_v2 while dag_v1 is still running).
- If a bug is introduced in the new DAG, I can quickly roll back to the previous version without downtime.
- It gives clear traceability — I can see which version of the DAG produced which output, which is important for debugging and compliance.
- Teams can review changes (using Git history, pull requests) before they go live.

In real projects, I keep DAGs in a Git repository. Every DAG change is a commit, and deployment is done via CI/CD. This way, I know exactly which code is in production, and I can reproduce old runs if needed.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

5. How can retries and SLAs be configured at the DAG level?

Retries and SLAs are part of making DAGs reliable.

- **Retries**: At the DAG or task level, I can set parameters like retries, retry_delay, and max_retry_delay. For example:

```
default_args = {
 'retries': 3,
 'retry_delay': timedelta(minutes=5),
}
```

This means if a task fails, Airflow will wait 5 minutes and retry up to 3 times. This is useful for transient failures like network errors or API timeouts.

- **SLAs (Service Level Agreements)**: I can set an SLA for a task to say "this must finish within X time." For example:

```
task = PythonOperator(
task_id='my_task',
python_callable=my_function,
 sla=timedelta(hours=1),
)
```

If the task runs longer than 1 hour, Airflow will log an SLA miss, and it can send email alerts. At the DAG level, this helps me monitor whether pipelines are meeting their deadlines.

So, retries handle transient failures automatically, and SLAs give visibility into whether pipelines are meeting business timelines.

6. Which Composer operator would you use to run a BigQuery SQL query?

To run a BigQuery SQL query in Composer (Airflow), I would use the **BigQueryOperator**.

For example:

```
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator


query_task = BigQueryInsertJobOperator(
  task_id='run_bq_query',
  configuration={
    "query": {
      "query": "SELECT COUNT(*) FROM my_dataset.my_table",
      "useLegacySql": False,
    }
  }
)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

In older versions, people often used BigQueryOperator, but now the recommended one is BigQueryInsertJobOperator, which is more flexible and matches how BigQuery jobs are configured.

This operator allows me to run queries, load jobs, or export jobs. For analytics pipelines, I mostly use it to run SQL transformations inside BigQuery as part of the DAG.

7. How can you orchestrate a Dataflow job using Cloud Composer?

In Composer, I can orchestrate Dataflow jobs using the **Dataflow operators** that Airflow provides. These operators allow me to trigger a Dataflow job as a task inside my DAG. For example:

- If I want to run a Python-based pipeline, I can use DataflowCreatePythonJobOperator.
- If I have a Java JAR, I can use DataflowCreateJavaJobOperator.
- For templates (flex or classic), I can use DataflowTemplatedJobStartOperator.

A typical DAG task might look like this:

```
from airflow.providers.google.cloud.operators.dataflow import
DataflowTemplatedJobStartOperator


dataflow_task = DataflowTemplatedJobStartOperator(
  task_id="start_dataflow_job",
  template="gs://my-bucket/templates/my_template",
  parameters={"input": "gs://data-bucket/input", "output": "gs://data-bucket/output"},
  location="us-central1"
)
```

Composer schedules and monitors the Dataflow job. If the job fails, the task fails and retries according to DAG settings. This orchestration ensures that Dataflow pipelines run as part of a larger workflow, like after a file lands in GCS or before loading results into BigQuery.

8. How can Composer interact with Cloud Storage buckets in a DAG?

Composer can interact with GCS using the **GCS operators and hooks** provided in Airflow's GCP provider. For example:

- GCSCreateBucketOperator to create a new bucket.
- GCSListObjectsOperator to list files in a bucket.
- GCSToBigQueryOperator to load files directly into BigQuery.
- LocalFilesystemToGCSOperator to upload local files into a bucket.

For example, to upload a file:

```
from airflow.providers.google.cloud.transfers.local_to_gcs import
LocalFilesystemToGCSOperator
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
upload_task = LocalFilesystemToGCSOperator(
  task_id="upload_to_gcs",
  src="/tmp/myfile.csv",
  dst="data/myfile.csv",
  bucket="my_bucket"
)
```

Composer also uses GCS sensors like GCSObjectExistenceSensor to wait until a file arrives before proceeding. This is very useful for event-driven DAGs.

So, GCS operators and sensors make it easy for Composer to integrate with storage workflows like staging, archiving, or triggering pipelines when new data arrives.

9. What is a Hook in Airflow, and when would you use it?

A Hook in Airflow is a reusable piece of code that manages the connection and interaction with an external system. It wraps the API or client library of that system so operators and sensors can use it easily.

For example:

- BigQueryHook connects to BigQuery and runs queries.
- GCSHook manages uploads and downloads from Cloud Storage.
- MySqlHook manages database connections.

I would use a Hook when I need to build an operator or sensor that talks to an external service. Instead of rewriting authentication and API logic every time, I call the Hook. Hooks also use Airflow's connection system, so I don't need to hardcode credentials.

So, Hooks are the building blocks under the hood — Operators use them, and I can use them directly if I need custom interactions.

10. Why might you create a custom operator for a pipeline?

I'd create a custom operator when my pipeline has logic that isn't covered by the built-in operators or when I want to simplify complex, repetitive steps into a reusable task.

For example:

- If my pipeline needs to call an internal REST API with specific authentication and parsing, I can write a CustomApiOperator.
- If I always perform the same sequence — like checking a file in GCS, validating schema, and then loading it into BigQuery — I can wrap that into one custom operator so the DAG stays clean.
- If I need special error handling, logging, or metrics that aren't part of the standard operators, I can extend an existing operator with my logic.

The benefit is reusability and maintainability. Instead of copying the same Python code across multiple DAGs, I package it into an operator and just call it in each DAG.

So, I'd create a custom operator to make pipelines cleaner, reduce duplication, and enforce standards across the team.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

11. How can Composer automate running SQL queries in BigQuery?

Composer (Airflow) can run BigQuery SQL as part of a DAG using BigQuery operators or hooks, so you treat SQL execution as a scheduled, monitored task in your workflow. Common ways I use in production:

- Use BigQueryInsertJobOperator (recommended newer operator) to submit a query job. It lets you pass the full job configuration (query, destination table, useLegacySql=false, priority, etc.).
- Use BigQueryExecuteQueryOperator / BigQueryOperator in older setups if available in your provider version — they provide simpler interfaces for one-off queries.
- For custom logic or dynamic SQL, use BigQueryHook inside a PythonOperator to run queries programmatically and fetch results (useful when you need to branch on query output).
- Use Jinja templating inside DAGs to inject dates / partitions into SQL at run-time ({{ ds }}, {{ ds_nodash }}, variables). This makes daily/partitioned queries easy to parameterize.
- Capture results via XCom (if the operator returns rows) or write results to a BigQuery staging table that downstream tasks read from.

Practical example snippet (composer DAG task):

```
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator


run_bq = BigQueryInsertJobOperator(
  task_id='run_report_query',
  configuration={
    "query": {
      "query": "SELECT user_id, COUNT(*) AS cnt FROM `proj.ds.events` WHERE event_date = @ds GROUP BY user_id",
      "useLegacySql": False,
      "queryParameters": [
        {"name": "ds", "parameterType": {"type": "DATE"}, "parameterValue": {"value": "{{ ds }}"}}
      ],
      "destinationTable": {"projectId":"my-proj","datasetId":"analytics","tableId":"daily_user_counts"},
      "writeDisposition": "WRITE_TRUNCATE"
    }
  },
  location='US'
)
```

Best practices I follow: run queries as BigQuery jobs (not ad-hoc API calls), use parameterization and templates for idempotency, prefer load jobs from GCS for large data movement, use service accounts with least privilege, and add monitoring/alerts on job failures or SLA misses.

© Shubham Wadekar

12. Which operator can be used to load data from GCS to BigQuery?

The go-to operator is GCSToBigQueryOperator (part of airflow.providers.google.cloud.transfers.gcs_to_bigquery). It directly runs a BigQuery load job from files sitting in GCS and exposes common options like schema, source format, write disposition, and partition decorators.

Example usage:

```
from airflow.providers.google.cloud.transfers.gcs_to_bigquery import GCSToBigQueryOperator


load_task = GCSToBigQueryOperator(
    task_id='load_gcs_to_bq',
    bucket='my-staging-bucket',
    source_objects=['data/{{ ds }}/events-*.avro'],
    destination_project_dataset_table='my_proj.analytics.events${{ ds_nodash }}',  # optional partition decorator
    source_format='AVRO',
    write_disposition='WRITE_TRUNCATE',
    create_disposition='CREATE_IF_NEEDED',
    autodetect=True,
    time_partitioning={'type': 'DAY', 'field': 'event_date'},  # if using column partitioning
)
```

Notes & best practices:

- For high-volume daily loads, write files to GCS first, then trigger a single load job via this operator — it's cheaper and atomic compared to streaming inserts.
- Use partition decorators (e.g., table$YYYYMMDD) or time_partitioning to load directly to a specific partition and avoid full-table scans.
- Provide a schema or use autodetect carefully (explicit schema is safer).
- Use write_disposition (WRITE_TRUNCATE, WRITE_APPEND, WRITE_EMPTY) depending on whether you want to overwrite or append.
- Make sure Composer service account has bigquery.dataOwner/bigquery.dataEditor on target dataset and storage.objects.get on GCS.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

13. How would you manage daily loads into a partitioned BigQuery table?

For daily loads into a partitioned BigQuery table, I would first design the table with partitioning based on the date column, usually an ingestion date or event date. This allows me to insert only the new data into the right partition without disturbing older partitions.

From an orchestration perspective, I would set up a Composer DAG that runs daily, pulling data from the source, transforming it if required, and then loading it into the specific partition. BigQuery makes this efficient because I can use the PARTITION BY clause to automatically place rows in the correct partition.

To optimize, I would:

- Use an external staging area (like Cloud Storage or a staging table) before merging to production.
- Ensure deduplication logic, since daily data might overlap with previous runs.
- Use append-only if data is strictly new, or a merge strategy if I need to update existing records.
- Add monitoring on load size and errors so I can identify issues quickly.

This way, every day's load is isolated into its partition, and queries on recent data run much faster and cheaper.

14. What is the difference between full refresh and incremental BigQuery loads in Composer?

A full refresh means that each time the pipeline runs, it deletes or overwrites the target table and reloads the complete dataset from the source. This approach is simple but costly and slow when datasets are large, and it is only practical if the dataset is relatively small or when changes are unpredictable.

An incremental load means that only new or changed data since the last load is processed and added to the target table. In BigQuery with Composer, I would usually do this by keeping track of a watermark column like last_updated or event_date, so only the latest records are fetched and inserted. Incremental loads are more efficient because they reduce data movement, save cost, and speed up the pipeline.

In real projects, I prefer incremental loads for large production datasets, and I use full refresh only for small lookup tables or when I need a clean rebuild after schema or business logic changes.

15. How do you handle schema changes when orchestrating BigQuery pipelines?

Schema changes are common in production pipelines, so I plan for them in advance. In BigQuery, if the schema evolves, I usually handle it in these ways:

- If it's an additive change (like new columns), I can allow BigQuery to relax the schema and load data without issues. My Composer pipeline can be configured with schemaUpdateOptions so new fields are added automatically.
- If columns are renamed or types are changed, I set up a staging layer first. The pipeline loads raw data into a staging table, then I apply transformations to map it to the production schema. This way, I have control over backward compatibility.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

- For dropping columns, I don't directly remove them from production tables because downstream jobs might break. Instead, I deprecate columns gradually and communicate with teams before physically removing them.
- I also keep schema definitions version-controlled, so any changes can be tracked.

In Composer, I usually add validation steps in the DAG to compare the source schema with the target before loading, so I can catch issues early. This makes the pipeline more robust when schema changes happen unexpectedly.

16. How do you trigger a batch Dataflow job from Composer?

To trigger a batch Dataflow job from Composer, I use the Dataflow operators available in Airflow. For batch pipelines, I usually build a Dataflow template first. This template is stored in GCS and contains the pipeline logic. Then, in my Composer DAG, I use the DataflowTemplatedJobStartOperator to launch that template.

The DAG specifies details like the project ID, region, template path in GCS, and runtime parameters. When the DAG runs, Composer calls the Dataflow service and starts the job. If the job is not templated, I can also trigger a flex template or even use the DataflowCreatePythonJobOperator to run the pipeline code directly.

In practice, I prefer templates because they make the deployment process consistent, and Composer only needs to supply parameters, not the full pipeline code.

17. How can you pass parameters to a Dataflow template in Composer?

When triggering a Dataflow template, Composer allows me to pass parameters as a dictionary. In the DAG, I specify a parameters field inside the Dataflow operator. Each parameter key matches the parameter name defined in the Dataflow pipeline, and the values are provided dynamically.

For example, I can pass the input path in GCS, output BigQuery table, or the date for processing. These can also be templated using Airflow macros, like {{ ds }} for the execution date, so each run gets a different parameter automatically.

This makes the pipeline flexible. Instead of hardcoding file paths or table names, I just pass them at runtime, and the same template can be reused for different datasets or schedules.

18. How does Composer monitor the status of a running Dataflow job?

Composer integrates with Dataflow through its operators, so it automatically tracks the job status. When I trigger a job with the Dataflow operator, Composer keeps polling the Dataflow service to check whether the job is running, finished, or failed.

If the job fails, the operator raises an exception and marks the DAG task as failed. This lets me set retries, alerts, or fallback logic in Composer. If the job succeeds, the task is marked successful and the pipeline continues.

In addition, I can push Dataflow logs to Cloud Logging, so I can see detailed job-level logs. Composer's task instance UI also shows the job ID, which I can click to directly open the Dataflow job in the GCP console.

So overall, Composer acts as the orchestrator and monitoring layer, while Dataflow executes the heavy processing.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

19. What happens if a Dataflow job triggered by Composer fails?

If a Dataflow job triggered by Composer fails, the task in the Composer DAG that started the job will also fail. Composer operators for Dataflow continuously monitor the job status. As soon as Dataflow reports that the job has ended with a failure state, the operator raises an error and marks that task as failed.

Because of this, the DAG can stop execution or move to any downstream tasks configured with failure handling (like retries, alerts, or branching). In real projects, I usually configure retries with exponential backoff in the operator, so temporary issues don't cause the whole pipeline to fail.

For deeper troubleshooting, the job logs are available in Cloud Logging, and Composer gives me the job ID in the task logs so I can open the Dataflow console directly. If the failure is due to bad data, schema mismatch, or quota issues, I might rerun the job after fixing the problem.

So, Composer doesn't silently ignore a failed Dataflow job—it reflects that failure in the DAG, which is important for reliability and monitoring.

20. What is the difference between classic templates and Flex templates in Dataflow orchestration?

Classic templates are the older way of packaging Dataflow jobs. With them, the pipeline code is compiled once, and a fixed template JSON file is stored in GCS. When Composer triggers the job, it can only pass runtime parameters like input path or date, but the pipeline logic itself cannot change. This makes classic templates good for stable pipelines but limited when you need more flexibility.

Flex templates are the newer and more powerful option. Instead of just a JSON file, Flex templates package the pipeline code inside a Docker container image. This container is stored in Artifact Registry, and when the job runs, Dataflow uses that container to launch the pipeline. Because of this, Flex templates support more dynamic behavior: you can use different versions of dependencies, custom code, or even updated pipeline logic without rebuilding the entire Dataflow deployment process.

In orchestration with Composer:

- Classic templates are triggered using DataflowTemplatedJobStartOperator.
- Flex templates are triggered with DataflowStartFlexTemplateOperator.

In real-world use, I prefer Flex templates because they're more portable, support custom dependencies, and are easier to manage in CI/CD. But for simple, stable pipelines, classic templates are still sufficient.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

21. How can Composer submit a Spark job to Dataproc?

Composer can submit a Spark job to Dataproc by using the Dataproc operators in Airflow. The usual operator is DataprocSubmitJobOperator. In the DAG, I define the cluster name, the type of job (like Spark, PySpark, or Hadoop), and the arguments such as the main JAR file or Python script along with input and output locations.

When the DAG runs, Composer makes an API call to Dataproc and submits the job to the cluster. If the cluster doesn't already exist, I can first create it using the DataprocCreateClusterOperator, run the Spark job, and then delete the cluster using DataprocDeleteClusterOperator.

In practice, this is very useful because I can orchestrate the whole lifecycle: cluster creation, job submission, monitoring, and cluster deletion — all controlled by Composer.

22. What is the benefit of orchestrating ephemeral Dataproc clusters in Composer?

The main benefit of orchestrating ephemeral Dataproc clusters is cost efficiency and flexibility. An ephemeral cluster is created only when the job is about to run and deleted right after the job completes.

This means I don't have to keep a cluster running 24/7 and paying for idle resources. Instead, I only pay for the duration of the job. It also allows me to configure the cluster each time with the exact hardware or software setup I need, so I can scale resources up or down depending on the workload.

Using Composer to orchestrate this ensures the process is automated — the DAG creates the cluster, runs the job, monitors it, and then deletes the cluster. This avoids manual intervention and prevents forgotten clusters from generating unnecessary costs.

23. Why might you delete a Dataproc cluster automatically after job completion?

I would delete a Dataproc cluster automatically after job completion mainly to save costs. Dataproc clusters run on Compute Engine VMs, and leaving them idle still incurs charges. If a cluster is not deleted, I could end up paying for machines that are doing nothing.

Another reason is resource management. By cleaning up clusters automatically, I avoid clutter in my environment and reduce the risk of hitting quotas on VM instances or persistent disks.

It also helps from a security and governance perspective. Ephemeral clusters don't stick around long, so there's less chance of unauthorized access to data or misconfigurations being left open.

In short, automatic deletion ensures I only pay for what I use, keeps the environment clean, and enforces good operational practices.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

24. How would you orchestrate a pipeline that runs Dataproc first and then writes to BigQuery?

I would design the Composer DAG so that the steps are clearly ordered. First, I would use a DataprocSubmitJobOperator to run the Spark or PySpark job on Dataproc. This job would handle the heavy transformation logic, for example reading raw data from GCS, applying transformations, and writing the output back to GCS in a structured format like Parquet or Avro.

Once that task completes successfully, the next task in the DAG would be a GCSToBigQueryOperator or BigQueryInsertJobOperator. This step loads the transformed data from GCS into the target BigQuery table.

By chaining these tasks, Composer ensures that the BigQuery load only starts after the Dataproc job is complete. I can also add retries, SLA alerts, and validation tasks in between, such as schema checks, before loading to BigQuery.

This orchestration gives me an end-to-end flow: Dataproc for heavy processing, and BigQuery for analytics-ready storage.

25. How can Composer help optimize Dataproc costs?

Composer can optimize Dataproc costs by automating the creation and deletion of clusters. Instead of keeping a Dataproc cluster running all the time, I can configure the DAG to create an ephemeral cluster only when the job needs to run and delete it afterward. This way, I only pay for the compute resources during the job's runtime.

I can also use Composer to dynamically size the cluster depending on the workload. For example, for small daily jobs, I can spin up a small cluster, and for monthly heavy jobs, a bigger cluster. This flexibility avoids over-provisioning.

Another cost optimization is scheduling. Composer can ensure Dataproc jobs run during off-peak hours if costs are lower, or coordinate workloads to avoid overlapping resource usage.

So, the main benefit is that Composer removes the risk of forgotten or idle clusters and gives full automation around cost-efficient cluster usage.

26. How can Composer trigger a pipeline when a new file arrives in GCS?

Composer can trigger a pipeline when a new file arrives in GCS by using sensors. The most common option is the GCSObjectExistenceSensor or GCSObjectUpdateSensor. These sensors keep checking if a particular file or a pattern of files has landed in the bucket. Once the file is detected, the downstream tasks in the DAG start running.

Another way is to use event-driven orchestration. GCS can publish a notification to Pub/Sub when a file is uploaded. Composer can use a PubSubPullSensor to listen for these messages and then trigger the pipeline.

Between the two, using Pub/Sub notifications is more efficient because sensors involve polling, while Pub/Sub is event-based and near real-time.

In practice, I would choose Pub/Sub + Composer if I need immediate triggering, and a GCS sensor if a small delay is acceptable and the pipeline runs periodically anyway.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

27. What operator would you use to move files from one GCS bucket to another?

I would use the Google Cloud Storage operators provided by the Airflow Google provider. Concretely, the common choices are the GCSToGCS/GCSCopyObject family (names can vary slightly by provider version) to copy or move objects between buckets. These operators let you specify source bucket/path, destination bucket/path, and rename rules or wildcard patterns.

If I need a true "move" (copy then delete), I either use the operator's built-in move/delete_source_objects option if available, or run a small two-step flow: 1) copy the objects with the GCS copy operator, then 2) delete the originals with GCSDeleteObjectsOperator (or a PythonOperator using GCSHook).

I prefer the managed operators because they handle retries, auth via Composer's connection, and work nicely with Airflow templating ({{ ds }}, file patterns). For more complex logic (filtering, checksum-based moves) I'd implement a small PythonOperator that uses the GCSHook so I have full control.

28. How can you validate schema for GCS files before loading into BigQuery?

I treat schema validation as an explicit pre-load step in the DAG. Practical ways I use:

- Keep a canonical schema definition (JSON or YAML) in repo or config. Before loading, use a PythonOperator that reads a sample of the file(s) from GCS and compares columns, types, and required fields against that canonical schema. If anything mismatches, fail the task and alert.
- Prefer binary formats with embedded schema like Avro or Parquet for production pipelines — those carry types, so BigQuery loads are safer and validation is simpler.
- For CSV/JSON files, check header presence, column count, and run lightweight type checks (e.g., sample rows parsed by pandas or pyarrow) to ensure numeric/date columns parse correctly and nullability is as expected.
- Use BigQuery load-job settings cautiously: you can enable schema auto-detection for ad-hoc loads, but in production I avoid it — I supply schema_fields or a schema JSON so BigQuery rejects bad inputs rather than silently misinterpreting types.
- Optionally, run a staging load into a temporary table and run SQL validation (row counts, uniqueness, constraint checks) before merging into the final table. That pattern is handy for late-arriving records or complex transformations.

In short: validate early (in Composer), prefer typed formats, and fail-fast so bad files don't contaminate BigQuery.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

29. How do you handle partitioned data when ingesting from GCS to BigQuery?

I design ingestion with partitions in mind so queries stay cheap and fast. Key practices I follow:

- Use a partition column: either ingestion time (_PARTITIONTIME / ingestion-time partitioning) or a logical date column (like event_date) and set timePartitioning.field in the BigQuery load job. That ensures rows land in the correct partition automatically.
- Use partition decorators if loading single-day files: load directly to dataset.table$YYYYMMDD to target a specific partition. This is useful for daily DAGs.
- For files that may affect multiple partitions or arrive late, I load into a staging table first, then run a SQL MERGE or INSERT INTO the partitioned table using the partition field to route rows. This lets me dedupe and handle late data without rewriting partitions.
- Keep partition size sane: if a partition becomes huge, consider further dividing with clustering (e.g., cluster by user_id) or change partition granularity (daily vs hourly) depending on query patterns.
- For performance: when writing from Composer, set the load job to WRITE_APPEND for simple daily appends, or use atomic MERGE for updates. Also validate that requirePartitionFilter is set on production tables to avoid accidental full-table scans by queries.

Overall, I make ingestion partition-aware (either direct partition load or staged merge) and add safeguards so bad loads don't touch historical partitions.

30. What file formats can be orchestrated in GCS pipelines with Composer?

Composer (Airflow) can orchestrate any file format you can store in GCS — the practical list I work with is:

- CSV / TSV (comma/pipe/tab-separated)
- Newline-delimited JSON (NDJSON) and JSON arrays (but NDJSON is preferred for BigQuery)
- Avro (great because it carries its schema)
- Parquet (columnar, efficient for analytics)
- ORC (another columnar option)
- Compressed variants of the above (gzip, bzip2) — just check the loader supports the compression type
- Google Sheets (can be accessed via the Sheets API and scripted in DAGs)
- Binary blobs (images, logs) — if you need to process them with custom code or Dataflow jobs

When loading into BigQuery specifically, BigQuery natively supports CSV, JSON (newline-delimited), Avro, Parquet, and ORC. For production pipelines I prefer Parquet or Avro for performance and schema safety. Composer's role is format-agnostic — it triggers the correct operator or transformation (Python, Dataflow, Spark) to parse whatever format you choose.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

31. How can Composer be triggered by a Pub/Sub message?

There are two practical ways I use in production: pull-based (sensor) and event-driven (push -> DAG run).

- Pull-based (sensor): I add a Pub/Sub sensor task in a DAG (for example PubSubPullSensor or PubSubSubscriptionSensor depending on Airflow version). The sensor polls a subscription and waits until it finds a message. Once the sensor succeeds, downstream tasks run with the message content passed via XCom or saved to GCS. This is simple to implement but is polling-based and can be wasteful if you need real-time behavior.
- Event-driven (recommended for real-time and scale): I configure Pub/Sub to push to a small Cloud Function or Cloud Run service. That function receives the Pub/Sub message and calls Composer's Airflow REST API to create a DAG run (/dags/<dag_id>/dagRuns) and passes the message as conf (JSON). The DAG reads dag_run.conf to access the payload and acts on it. This is near-real-time, scales well, and avoids continuous polling inside Composer. I secure the call with a service account that has minimal privileges.

In interviews I emphasize: use sensor for simple setups or where polling is acceptable; use push-to-function + DAG run for production-grade, low-latency triggers.

32. How would you orchestrate a workflow that processes Pub/Sub streaming data daily in batch?

I split the system into two layers: streaming ingestion and daily batch processing.

1. Streaming ingestion: run a long-lived streaming Dataflow job that subscribes to Pub/Sub, does light parsing/deduplication, and writes raw events into a durable, partitioned store — typically GCS (Parquet/Avro) or a raw BigQuery table partitioned by event date. The streaming job focuses on durability and minimal transformations so it can keep up with event velocity.
2. Daily batch (Composer): create a Composer DAG scheduled once per day. The DAG checks that the day's partition (or GCS folder) is ready (via a manifest, sensor, or Pub/Sub event), then launches a batch job (Dataproc or Dataflow batch) to read that day's raw files, perform heavy joins/enrichment/aggregation, and write results to a partitioned BigQuery table. After loading, the DAG runs validation (row counts, sanity checks) and a MERGE if dedup/upsert is needed.
3. Late data & backfills: have a parameterized reprocess DAG that can rerun for specific dates if late messages arrive. Also use watermarking in the streaming job and write late events to a special area that the daily batch can pick up or a backfill process consumes.

This pattern gives low-latency ingestion plus a single daily authoritative batch run for heavy computation and reporting.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

33. How can you handle hybrid batch + streaming pipelines in Composer?

I handle hybrids by treating streaming as the ingestion layer and batch as the completeness/analytics layer, then using Composer as the central orchestrator and control plane:

- Separate concerns: keep a long-running streaming job (Dataflow) for real-time ingestion and short-lived batch jobs (Dataproc/Dataflow batch) for heavy transforms and backfills. Composer schedules and monitors batch jobs and can deploy or restart streaming jobs when needed.
- Use a common canonical store: make streaming write raw events to a partitioned store (GCS/BigQuery). Batch jobs read the same store to produce consistent, deterministic outputs. This avoids duplication of logic and gives a single source of truth.
- Make jobs idempotent and dedupable: design streaming and batch code to use event IDs and MERGE semantics or dedupe logic so reprocessing does not corrupt data.
- Coordinate with Composer:
    - Trigger batch runs when partitions are complete (sensors or Pub/Sub triggers).
    - Provide DAGs for backfills and reprocessing specific date ranges.
    - Automate deployments of streaming templates (Flex templates) and restarts from Composer if streaming jobs fail.
- Monitoring and SLAs: include checks in Composer for streaming health (lag, watermark) and configure alerts; have the DAG fail fast on anomalies so operators can act.

In short: streaming = fast ingestion and durability, batch = heavy, consistent computation; Composer = automation, coordination, recovery and visibility across both.

34. What is the purpose of a dead-letter topic when orchestrating Pub/Sub pipelines?

A dead-letter topic is used as a safety net when messages cannot be processed successfully. In Pub/Sub pipelines, if a subscriber or a downstream job (like Dataflow) fails to acknowledge a message after multiple retries, instead of dropping the message, Pub/Sub moves it to the dead-letter topic.

The main purpose is to avoid data loss. Messages that are bad, corrupted, or causing repeated failures get isolated in this separate topic. Later, I can build another Composer DAG or Dataflow job to inspect those dead-letter messages, fix the root cause (like schema mismatch or bad data), and reprocess them.

This improves reliability of the whole system because the main pipeline continues to work without being stuck on bad records, and I still have a mechanism to recover or analyze the failed ones.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

35. How can Composer coordinate Pub/Sub ingestion with Dataflow → BigQuery pipelines?

Composer acts as the orchestrator that ties everything together. A common setup is:

1. Pub/Sub is the entry point where events arrive.
2. A long-running Dataflow streaming job subscribes to Pub/Sub and ingests events. This Dataflow job performs parsing, enrichment, and writes data to BigQuery (usually into a raw or staging table).
3. Composer is responsible for deploying and managing that Dataflow job. In my DAG, I can use operators like DataflowStartFlexTemplateOperator to start the streaming pipeline or to restart it if it fails.
4. Composer can also schedule downstream batch jobs that read the staging BigQuery tables (populated via Pub/Sub → Dataflow) and then perform daily aggregations, partition merges, or move curated data into analytics tables.

So Composer doesn't directly ingest from Pub/Sub but makes sure the Dataflow job is running, healthy, and followed up by the right BigQuery processing. It provides scheduling, retries, monitoring, and coordination across all components.

36. How can Composer orchestrate a multi-step ETL pipeline?

Composer is designed for multi-step pipelines because it lets me break the ETL into ordered tasks in a DAG. For example:

1. Extract: first task pulls raw data from sources like GCS, APIs, or databases. I can use operators such as GCSToGCSOperator, MySqlToGCSOperator, or even custom PythonOperators.
2. Transform: next tasks process and clean the data. This can be a Spark job on Dataproc, a batch Dataflow template, or even BigQuery SQL transformations. Composer triggers these jobs, waits for them to finish, and moves to the next step.
3. Load: final tasks load the transformed data into the target system, most often BigQuery. Operators like GCSToBigQueryOperator or BigQueryInsertJobOperator handle this part.
4. Validation: I usually add checks, like row counts or schema validation, before marking the DAG as complete.
5. Notifications: optional steps send success/failure alerts via email, Slack, or Pub/Sub.

The benefit is that Composer lets me clearly define dependencies between steps (load happens only if transform succeeds, etc.), manage retries for each task, and monitor the pipeline end-to-end. It turns a complex ETL flow into a reliable, automated DAG that can run daily, hourly, or on-demand.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

37. What is the difference between ETL vs ELT in Composer workflows?

ETL means Extract → Transform → Load. In this style, I first pull the raw data out of the source (Extract), then apply transformations outside the warehouse, usually using Dataflow, Dataproc, or Spark (Transform), and finally load the cleaned and shaped data into BigQuery (Load). Composer orchestrates this by chaining these steps as separate tasks in the DAG. This approach is useful if transformations are heavy, require distributed compute, or if the warehouse should only store clean data.

ELT means Extract → Load → Transform. In this style, I extract raw data and directly load it into BigQuery first. Once data is inside BigQuery, I run SQL transformations there to clean, join, and aggregate. Composer orchestrates this by first running load tasks, then scheduling SQL queries on BigQuery. This approach is more modern because BigQuery has strong processing power, and it reduces complexity of external systems.

So the difference in Composer is mainly the order of tasks: ETL workflows include external transformation jobs before BigQuery, while ELT workflows load raw data straight into BigQuery and rely on Composer to trigger SQL transformations after.

38. How would you orchestrate incremental loads in a pipeline with Composer?

For incremental loads, the idea is to only process new or changed data instead of reloading everything. In Composer, I usually handle this with a parameter like a date or a high-watermark column (such as last_updated timestamp or an auto-incrementing ID).

The DAG steps are:

1. Extraction task queries the source system with a filter like WHERE last_updated > <last_run_time>. I can pass this value dynamically using Airflow macros (for example {{ ds }} for the execution date).
2. The pipeline then moves the incremental data into a staging area (GCS or a staging BigQuery table).
3. Transformation jobs clean and deduplicate.
4. Finally, I merge into the target BigQuery table using a MERGE SQL statement. This way, new rows are inserted and existing rows are updated.

Composer makes this repeatable by scheduling it daily or hourly, and it ensures each step runs in order. I also add checkpoints (like storing the last successful watermark in Composer Variables or GCS) so if a job fails, I don't lose track of where to restart.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

39. How do you handle errors across multiple ETL steps in Composer?

In real pipelines, any ETL step can fail, so error handling is very important. I handle this in several ways:

- Task-level retries: Each task in Composer can be given retries with exponential backoff. If a transient error happens, the task retries before failing the whole DAG.
- Branching or fallback: I can design the DAG to branch on failure. For example, if transformation fails, I can send data to a quarantine bucket or trigger a notification task.
- Notifications: I always add failure alerting through email, Slack, or Pub/Sub so the team knows immediately when something breaks.
- Isolated stages: I design the DAG so each stage (extract, transform, load) is independent. This way, if one stage fails, I can rerun from that stage without rerunning everything.
- Staging areas: I use staging tables or buckets so failed transformations don't pollute final tables.
- Dead-letter handling: For record-level errors (like bad rows), I redirect them into a dead-letter table or file for later analysis.

So overall, Composer manages workflow-level reliability with retries and dependencies, and I add extra steps to handle data-level errors gracefully.

40. How are dependencies managed between extraction, transformation, and loading tasks?

In Composer, dependencies are managed explicitly in the DAG. Each task is a node, and I define which tasks must run before others. For example, I can say extract >> transform >> load. This ensures extraction finishes successfully before transformation starts, and transformation must finish before loading begins.

Airflow (which Composer is built on) uses Directed Acyclic Graphs, so the order is clear and cycles are not allowed. I can define dependencies either linearly (one after the other) or with branches and parallelism. For example, multiple extract tasks can run in parallel, and once all succeed, a transformation task starts.

Dependencies also help with failure management: if an upstream task fails, the downstream tasks are automatically skipped. This prevents bad data from flowing through the pipeline.

So dependencies in Composer make the ETL flow reliable, ordered, and easy to visualize. It's like drawing the ETL steps as a flowchart, where each step only runs when its prerequisites succeed.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

41. How can you schedule a DAG to run every day at midnight?

In Composer, a DAG is scheduled using the schedule_interval parameter. To run every day at midnight, I can simply set schedule_interval='0 0 * * *' which is a cron expression meaning "at 00:00 every day." Another simpler way is to use the preset '@daily', which also means once a day at midnight UTC.

So a DAG definition would look like:

- schedule_interval='0 0 * * *' or
- schedule_interval='@daily'

If I want it to run at local time midnight instead of UTC, I adjust the cron to match my local timezone or configure the DAG's timezone.

42. What is the difference between start_date and schedule_interval in Composer DAGs?

The start_date tells Airflow (and Composer) the first date and time when the DAG is allowed to start running. It's like saying "the DAG becomes active from this date onward." If the current date is before the start_date, the DAG won't run.

The schedule_interval defines how often the DAG should run after the start date. For example:

- If start_date=datetime(2023,1,1) and schedule_interval='@daily', then the DAG will try to run once a day starting from Jan 1, 2023 at midnight.

So in short:

- start_date = when the DAG can start scheduling from.
- schedule_interval = how frequently the DAG should run after the start.

They work together to control timing.

43. What does catchup=True mean in a DAG?

catchup=True means that when you enable or backfill a DAG, Composer will run all the past scheduled runs between the start_date and the current date.

For example, if I set start_date=2023-01-01, schedule_interval='@daily', and today is 2023-01-10, when I turn on the DAG with catchup=True, it will run 10 separate DAG runs (Jan 1 through Jan 10). This is useful when I want to process all missed periods of data, like backfilling partitions in BigQuery.

If catchup=False, then the DAG will only run from "now" onward and will skip all the missed schedules in the past. This is useful for streaming-like jobs or when I only care about current and future data.

So in interviews, I explain it as: catchup controls whether the DAG should backfill old runs or just start fresh from the current date.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

44. How can you backfill missed DAG runs in Composer?

In Composer, backfilling means running DAG instances for past dates that were missed. If my DAG has catchup=True, Composer will automatically create and run all the missed intervals between the start_date and today once I turn the DAG on. This is useful when I want to fill historical data partitions.

If I don't want automatic catchup, but I need to backfill manually, I have two options:

- Use the Airflow UI: I can go to the DAG runs page, pick past dates, and trigger them manually.
- Use the CLI: I can run a backfill command with gcloud composer environments run <env> --location <region> dags backfill -- <dag_id> -s <start_date> -e <end_date>. This forces DAG runs for that date range.

I usually use staging tables or flags to make sure backfilled data doesn't overwrite production until I validate it. This gives me flexibility to replay history safely.

45. How can SLAs be defined for data pipeline tasks in Composer?

In Composer, SLAs are like time promises for tasks. I can define an SLA by adding the sla parameter in the task. For example, if I expect a task to finish within 30 minutes, I can write sla=timedelta(minutes=30). Airflow will then track this, and if the task takes longer than expected, it marks it as an SLA miss.

SLA misses can trigger alerts. I can set up sla_miss_callback in the DAG to send an email, push a Pub/Sub message, or post to Slack. This way, my team knows if a task is running slower than it should.

Defining SLAs helps in real projects because business users often need guarantees, like "data must be available in BigQuery by 7 AM." By putting SLAs in Composer, I can monitor if I'm meeting those expectations.

46. How do you configure the number of retries for a task in Composer?

Retries are defined per task or globally in default_args. If I want a task to retry, I add the retries parameter, for example retries=3. I can also set a retry_delay, like retry_delay=timedelta(minutes=5), which means Composer will wait 5 minutes before trying again.

If I want to make retries smarter, I can enable exponential backoff, so each retry waits longer than the previous one, up to a max retry delay. This is useful if the failure is due to temporary issues like API quota or network glitches.

In real projects, I usually set retries in default_args so most tasks inherit the same policy, but I override them for special tasks. For example, I might set retries to zero for tasks that are not idempotent, like sending an email, but keep retries high for tasks like extracting data from GCS.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

47. How would you handle transient errors vs permanent errors in DAGs?

I treat them differently because their remedies are different.

For transient errors (network blips, temporary API rate limits, short GCS/BigQuery hiccups):

- Rely on retries: configure sensible retries, retry_delay, and retry_exponential_backoff so Airflow automatically retries the task a few times before failing. That usually clears transient problems.
- Make tasks idempotent: design tasks so retrying them doesn't cause duplicate side-effects (use upserts, temp staging, dedupe).
- Short circuit / backoff logic: for known transient patterns, add exponential backoff or small wait-and-check loops inside the task so the task itself can tolerate short outages.
- Monitor and alert only after retries are exhausted so SREs don't get woken for transient noise.

For permanent errors (bad schema, malformed input, permission denied, logic bugs):

- Fail fast and stop downstream work: let the task fail quickly so we don't waste compute or corrupt downstream data.
- Surface clear error messages and logs: include diagnostic info (bad file name, failing SQL, sample rows) so engineers can triage.
- Route bad records to dead-letter storage or staging tables if the issue is data-related, so you don't lose anything and can reprocess after fixes.
- Trigger remediation workflows: either a human-notification or a separate DAG that quarantines or attempts a controlled fix.

Operationally I make the DAG robust by using retries and idempotency for transient issues, and strict validation + fail-fast + quarantine for permanent issues so fixes are safe and observable.

48. How can you trigger a fallback task when a job fails?

There are a couple of clean patterns I use in Composer/Airflow:

- Downstream task with a failure trigger rule: create the fallback task and set its trigger_rule to one_failed (or all_failed depending on needs). That makes the fallback run only when at least one upstream task failed. Example flow: main_task >> fallback_task with fallback_task.trigger_rule = 'one_failed'.
- Use on_failure_callback: attach an on_failure_callback function to the task or DAG that runs custom logic (send detailed alerts, create tickets, publish a Pub/Sub message, trigger another DAG via API). This is good for notifications or kicking off external remediation.
- Branching: use a BranchPythonOperator or sensors that check previous task state via XCom / task instance state and then choose the fallback path if failure conditions are detected.
- Separate "watcher" DAG: have a small DAG that monitors critical task states (using the Airflow REST API or Cloud Logging metrics) and triggers fallback runs when failures are observed — useful if remediation should be managed by a different team or have separate retries.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

In interviews I usually show the trigger_rule='one_failed' pattern because it's simple, explicit, and lives inside the DAG, while on_failure_callback is great for external alerts and automated remediation.

49. How can alerts be set up for DAG failures?

I set up alerts on multiple levels so failures are visible quickly:

- Airflow built-in notifications:
  - email_on_failure=True and provide email in default_args for quick email alerts.
  - Use on_failure_callback for custom notifications (Slack webhook, PagerDuty, or Pub/Sub).
- Cloud-native integrations:
  - Export Airflow logs and metrics to Cloud Logging/Monitoring and create alerting policies (e.g., task failures > 0, DAG run failure, SLA misses). Those alerting policies can page SREs or send Slack/PagerDuty alerts.
  - Publish a Pub/Sub message on failure and subscribe Cloud Functions or Ops systems to that topic for automated processing.
- Task- and DAG-level callbacks:
  - on_failure_callback or DAG-level on_failure_callback to collect diagnostics (task logs, XComs, error context) and include them in the alert so engineers have immediate context.
- SLA alerts:
  - Define sla on critical tasks and implement sla_miss_callback to alert when a task misses its time budget.
- Runbook + escalation:
  - Tie alerts to an incident runbook (link in the alert) and use escalation rules (PagerDuty) so the right person is paged depending on time of day.

I prefer a layered approach: quick channels (Slack/email) for ops, pager for critical outages, and Cloud Monitoring for longer-term dashboards and automated triggers.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

50. When would you use a fail-fast strategy instead of continuing on error?

I choose fail-fast when continuing would make things worse or waste a lot of resources. Typical scenarios:

- Data integrity risk: if downstream processing would corrupt production tables (wrong schema, bad join keys, malformed records), fail-fast to prevent contamination and require human review or controlled reprocess.
- Critical dependency failure: if a required upstream dataset or credential is missing, there's no point continuing — fail and fix the dependency first.
- Expensive compute: if a failure early means heavy compute will be wasted (big Spark/Datapflow job that depends on clean input), fail-fast to avoid cost and repeated failures.
- Business SLAs and correctness: when business consumers expect accurate daily reports (e.g., financial close), a single wrong run must not produce outputs — fail and alert so humans decide whether to backfill or patch.

Contrast: continue-on-error (or resilient workflows) is better when partial outputs are acceptable, non-critical tasks should not block the pipeline, or fallback behavior (quarantine bad data) preserves progress.

In short: use fail-fast when correctness, cost, or downstream safety matter more than availability; use tolerant/continue behavior when partial progress and eventual consistency are acceptable.

51. What are XComs in Airflow, and how are they used in Composer?

XComs (short for "cross-communications") are a way for tasks in Airflow to share small pieces of information with each other. In Composer, every task runs in isolation, but sometimes you want to pass a value from one task to another, like a file path, row count, or query ID. That's where XComs are useful.

For example, a task that generates the GCS file name for today's data can push that file path into XCom. A downstream task can then pull that value from XCom and use it as input. In practice, I use methods like xcom_push and xcom_pull in PythonOperators, or rely on return statements from Python tasks, which automatically push values into XCom.

In short, XComs are great for sharing small metadata values between tasks in a Composer DAG without having to store them externally.

52. Why should large payloads not be passed using XComs?

XComs are stored in Airflow's metadata database, which is not designed for big data. Passing large payloads, like entire datasets or long JSON dumps, can cause problems:

- It slows down the metadata database and makes the web UI heavy.
- It creates performance issues because XComs were designed for small key-value metadata, not full data storage.
- In some cases, the size can exceed limits and cause failures.

That's why XComs should be used only for lightweight values like file names, dates, IDs, or row counts. For anything larger, I always use external storage.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

53. How can you store large datasets between tasks if not in XComs?

For large datasets, I always rely on external storage systems instead of XComs. Some common approaches I use:

- Write the intermediate dataset to GCS. One task writes its output to a GCS bucket, and the next task reads it from there. This is the most common pattern in data engineering pipelines.
- Use BigQuery staging tables. One task loads or transforms data into a staging table, and downstream tasks query or merge from it.
- Use Dataproc HDFS or Cloud Storage connectors if Spark jobs are involved.
- For structured key-value data, Pub/Sub can also be used to pass batches of events between tasks or jobs.

The idea is to let tasks exchange only a reference through XCom (like a GCS path or BigQuery table name), while the actual large dataset stays in scalable storage. This keeps Composer lightweight and makes pipelines more reliable.

54. How can you push and pull XComs in a PythonOperator?

In a PythonOperator you push and pull XComs using the task context or the ti (task instance) object. The common patterns I use:

- Push by returning a value from the Python callable (Airflow will store the return value as an XCom automatically). This is the simplest for small values (IDs, file paths, counts).
- Explicit push using ti.xcom_push(key, value) if I want to set a specific key.
- Pull with ti.xcom_pull(task_ids='upstream_task_id', key='my_key') or just ti.xcom_pull(task_ids='upstream_task_id') to get the return-value XCom.

Example (clear, interview-style):

```
from airflow.operators.python import PythonOperator


def produce(**context):
    ti = context['ti']
    path = f"gs://my-bucket/data/{context['ds']}.parquet"
    # implicit push by return:
    return path
    # or explicit:
    # ti.xcom_push(key='output_path', value=path)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def consume(**context):
    ti = context['ti']
    # pull the implicit return-value XCom
    path = ti.xcom_pull(task_ids='produce_task')
    # or pull by key:
    # path = ti.xcom_pull(task_ids='produce_task', key='output_path')
    print("Got path:", path)

produce_task = PythonOperator(
    task_id='produce_task',
    python_callable=produce,
    provide_context=True
)

consume_task = PythonOperator(
    task_id='consume_task',
    python_callable=consume,
    provide_context=True
)

produce_task >> consume_task
```

Keep in mind: XCom payloads should be small and JSON-serializable (or pickleable depending on Airflow config). For larger data, push only a reference (GCS path, table name) via XCom.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

55. What are best practices for sharing metadata between tasks in Composer?

I follow these rules in production:

- Share *small* metadata only: use XCom for tiny items — file paths, row counts, job IDs, status flags. Don't pass large blobs or raw data.
- Pass references, not data: store large outputs in GCS / BigQuery and pass the storage path or table name through XCom or Variables.
- Be explicit and consistent with keys: use clear XCom keys or well-known return-value conventions so downstream tasks know what to xcom_pull.
- Use JSON-serializable objects: keep XCom payloads plain JSON (dicts, strings, numbers). This avoids serialization surprises across Airflow versions.
- Avoid secrets in XComs: never put credentials or PII into XComs. Use Secret Backends, Airflow Connections, or Secret Manager instead.
- Use Airflow Variables or Connections for configuration: put environment-specific parameters (bucket names, dataset IDs) in Variables or Connections, not hard-coded in DAGs.
- Version and validate metadata: if you pass schema/version info, include a small version field so pipelines can detect incompatible changes and fail fast.
- Prefer TaskFlow / XComArg for clarity: when using the TaskFlow API, XComArg and function annotations make intent clearer and reduce manual xcom_pull boilerplate.
- Keep lifecycle and ownership clear: document what upstream task creates the value and which downstream tasks consume it; this avoids brittle DAG coupling.

Following these makes metadata passing predictable, debuggable and safe.

56. How can you use Airflow variables to configure runtime parameters?

Airflow Variables are my go-to for runtime config and small environment settings. Key points I mention in interviews:

- Where to set them: Variables can be created via the Airflow UI (Admin -> Variables), via CLI, or programmatically (and in Composer you can use gcloud composer environments run ... variables -- ... to set them).
- How to read them in code: use from airflow.models import Variable and then Variable.get("my_var"). You can provide a default: Variable.get("my_var", default_var="default"). For JSON configs use Variable.get("config", deserialize_json=True).
- Use-cases: bucket names, dataset/table IDs, feature flags, API endpoints, and thresholds. Keep sensitive items out of Variables unless you use a secret backend.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

- Best practices:
    - Treat Variables as environment-level config, not per-run parameters; for per-run values use DAG conf or dag_run.conf.
    - Store structured config as JSON and read with deserialize_json=True so tasks get a dict.
    - Avoid placing secrets in Variables. Instead integrate Airflow with Secret Manager or use Airflow Connections for credentials.
    - Use Variables in templated fields: {{ var.value.my_var }} or {{ var.json.my_json_config['key'] }} to inject them into operators.
    - Default and validation: code defensively—provide defaults and validate Variable contents at DAG start so bad config fails fast.

Example usage:

```
from airflow.models import Variable


bq_dataset = Variable.get("bq_dataset", default_var="my_dataset")

config = Variable.get("etl_config", deserialize_json=True)
```

This pattern gives flexibility: platform teams can change runtime behavior without touching DAG code, and developers can parameterize pipelines safely and cleanly.

58. How can you pass different configs for dev, test, and prod environments?

I separate environment config from DAG code so the same DAG can run in dev/test/prod with different settings. Common approaches I use:

- Airflow Variables per environment: create Variables named the same across environments (e.g., gcs_bucket, bq_dataset) with environment-specific values. The DAG reads Variable.get("gcs_bucket") and uses the correct bucket depending on the Composer environment it's deployed to.
- Use separate Composer environments: deploy the same DAG to three Composer environments (dev/test/prod). Each environment has its own Variables, Connections, and IAM scope, so code stays identical while config differs.
- Use DAG-level parameters + dag_run.conf: for ad-hoc runs or tests, pass overrides via dag_run.conf. This is handy for manual test runs without changing Variables.
- Use a small config file in GCS or Cloud Storage per environment: DAG reads gs://configs/<env>/pipeline_config.json where <env> is set via an environment variable or Airflow Variable. Useful when config objects are large.
- Use Airflow Connections and Secret Manager for credentials and sensitive values, not plain Variables.

Operational pattern I follow: keep environment selection out of code (e.g., ENV variable) and read all environment-specific values from Variables/Secrets so deployments are safe and repeatable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

59. How can parameterization make DAGs reusable?

Parameterization turns hard-coded values into inputs, so one DAG can handle many similar jobs. I make DAGs reusable by exposing parameters for things that change between pipelines: input path, output table, partition date, compute size, or even which transforms to run.

Ways I parameterize:

- Use default_args + DAG params for common defaults and allow overrides via dag_run.conf for one-off runs.
- Pass parameters into operators via templates or function args (TaskFlow API): @task functions accept inputs and return outputs, letting you reuse the same function with different parameters.
- Use Variables or a config JSON (read at runtime) so DAG logic reads config keys like pipeline['source'], pipeline['sink'].
- Provide sensible defaults and validate params at DAG start so bad runs fail fast.

Benefits in practice:

- One DAG can run for many tables/datasets just by changing parameters — fewer DAG files, less duplication, easier maintenance.
- Easier CI/CD: tests only need to change params to emulate other environments or datasets.

60. How would you design a DAG with different input/output paths dynamically?

I design it so that the DAG computes or receives the path values at runtime and passes only references between tasks. Typical patterns I use:

- Built from templates and Variables: combine Airflow macros with Variables:
  input_path = f"gs://{Var('raw_bucket')}/year={{{{ ds.split('-')[0] }}}}/month={{{{ ds.split('-')[1] }}}}/day={{{{ ds.split('-')[2] }}}}/file-{{{{ ds_nodash }}}}.parquet"
  Put that string into an operator field that supports templating so it resolves per run.
- Parameterized dag_run.conf: allow callers (or Pub/Sub trigger) to set {"input_path":"gs://...", "output_table":"proj.dataset.table"}. The DAG reads dag_run.conf at start and uses those values. This is ideal for ad-hoc or backfill runs.
- Compute in a small PythonOperator: if logic is more complex (discovering latest file, globbing GCS), I run a Python task that lists/validates files and then xcom_pushes the selected input_path. Downstream tasks xcom_pull that path and use it. Example flow: discover_files >> transform >> load where discover_files stores gs://.../file.parquet in XCom and transform pulls it.
- Use partition decorators for BigQuery loads: when writing to partitioned tables, construct the target as dataset.table${{ ds_nodash }} to target specific partitions dynamically.
- Keep idempotency and validation: always validate the dynamically chosen paths before heavy processing (check existence, size, schema) and write intermediate data to a staging path that includes a run-id or ds to avoid collisions.

This approach keeps the DAG flexible, safe, and easy to reuse across dates, environments, and datasets.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

61. How can you monitor DAG execution in the Composer UI?

In Composer, the Airflow UI is the main place to monitor DAG execution. When I open the Composer environment and go to the Airflow webserver, I see all my DAGs listed. For each DAG I can check:

- The status (success, running, failed, skipped) of each run in the "DAG Runs" page.
- The Tree View or Graph View to see task-by-task progress in a run. Green means success, red means failure, yellow means running.
- The Gantt chart to check task execution timelines and see which tasks are bottlenecks.
- The Duration view to track how long tasks usually take and detect anomalies.
- SLA misses or other metadata if I have them configured.

So, monitoring in the Composer UI is mostly visual — I can see how the pipeline is progressing and quickly drill into problem areas.

62. Where do you check task logs when a DAG fails?

When a DAG fails, the first step is to open the Airflow UI in Composer and click on the failed task. There is a "Log" button that shows the detailed logs of that task. These logs come from the worker pod or VM that ran the task.

If I need more context, Composer also sends all logs to Cloud Logging (formerly Stackdriver). I can go into the GCP console → Logging → Logs Explorer and filter by environment name, DAG ID, or task ID. This is helpful for long logs that are truncated in the UI, or if I want to set up monitoring queries on them.

So, for quick debugging I check the Airflow UI task logs, and for deeper analysis or history, I check Cloud Logging.

63. How can you configure email or Slack alerts for DAG failures?

There are multiple ways to set up alerts for DAG failures in Composer:

- Email: In default_args, I set email=['team@example.com'] and email_on_failure=True. This way, if any task fails, Airflow sends an email with details. Composer uses the configured SMTP backend, so I just need to make sure SMTP is set up.
- Slack: I use on_failure_callback on the DAG or task. This is a Python function that posts a message to Slack using a webhook or the Slack API. For example, I create a small Python function that formats the task ID, DAG ID, and error message, then sends it to the Slack channel.
- Cloud Monitoring: Since Composer sends logs to Cloud Logging, I can also create an alerting policy in Cloud Monitoring that triggers when it sees a "Task failed" log entry. That alert can send notifications via email, Slack, PagerDuty, or SMS.

In real projects, I usually combine email (for basic alerts) with Slack (for faster team visibility). For business-critical DAGs, I also set up Cloud Monitoring alerts to ensure failures are never missed.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

64. How can you track pipeline execution time for SLA monitoring?

In Composer, I can track execution time in a few ways. The simplest is to use the Airflow UI — it has a Gantt chart and a Duration view that show how long each task and DAG run took. This helps me see if tasks are running longer than usual.

For formal SLA monitoring, I define sla parameters on critical tasks. For example, I might say sla=timedelta(minutes=30). If the task takes longer than 30 minutes, Airflow marks it as an SLA miss. I can also set up an sla_miss_callback to alert the team when that happens.

Additionally, since Composer pushes logs and metrics into Cloud Logging and Cloud Monitoring, I can create dashboards and alerts to track DAG run duration and detect anomalies. This way, I get notified automatically if a DAG exceeds its normal runtime.

65. Why might you add custom logging in Composer DAGs?

I add custom logging in Composer DAGs so that debugging and monitoring become easier. By default, Airflow logs system messages, but that often isn't enough when I want to know what happened inside my pipeline.

For example:

- If I'm processing data files, I log how many rows were read, the file path, and schema details.
- If I'm loading into BigQuery, I log the destination table, partition, and number of rows inserted.
- If something fails, I log the input parameters and error details so I can reproduce the issue later.

Custom logging also helps when I integrate with monitoring systems. Since Composer logs flow into Cloud Logging, my custom logs can be searched, filtered, and even used to trigger alerts in Cloud Monitoring.

So overall, custom logging makes pipelines more transparent, easier to troubleshoot, and safer to run in production.

66. How can you use Git for version controlling DAGs in Composer?

Using Git for DAGs is a best practice because it provides history, collaboration, and rollback. Here's how I usually do it:

1. Keep all DAG files, plugins, and dependencies in a Git repository. Every change is committed and reviewed through pull requests.
2. Tag releases or branches for dev, test, and prod environments, so I can promote DAGs safely without surprises.
3. Use CI/CD pipelines (for example Cloud Build or GitHub Actions) to automatically deploy the latest DAGs from Git to the Composer environment's dags/ bucket in GCS. This way, no one edits files directly in GCS — all changes go through Git.
4. Store configurations (like environment-specific buckets and datasets) separately, so the same DAG code can work in multiple environments.
5. Add clear commit messages and documentation so I know exactly what changed in the pipeline when looking at history.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

This makes Composer pipelines auditable, easy to roll back, and consistent across environments.

67. How can you automate DAG deployment using Cloud Build?

I can set up Cloud Build as a CI/CD pipeline to automatically push DAGs to Composer whenever there's a code change. The flow works like this:

1. Store all DAGs in a Git repo.
2. Create a Cloud Build trigger connected to the repo (for example, on every commit to main or release branch).
3. In the cloudbuild.yaml file, add a step that copies the DAG files into the Composer environment's GCS bucket under the dags/ folder. This is usually done with gsutil cp commands.
4. Optionally, run linting or unit tests on the DAGs before deployment to catch errors early.
5. Cloud Build then automatically updates the DAGs in Composer without manual intervention.

This makes deployments consistent and ensures no one is manually editing DAG files in GCS. Every deployment is tied back to a Git commit for full traceability.

68. Why should DAGs be tested in dev before promoting to prod?

Testing in dev is important because Composer DAGs can affect production data, and mistakes can be costly. If I deploy directly to prod without testing:

- A bug in SQL or Python code might corrupt BigQuery tables.
- Wrong configurations could send data to the wrong bucket or dataset.
- Performance issues might delay critical reports or increase costs.

By testing in dev:

- I can validate logic, schema handling, and dependencies in a safe environment.
- I catch failures early without affecting production users.
- I can tune retries, SLAs, and resource settings before promoting.

Once the DAG runs clean in dev, I move it to test or staging with production-like data, and then finally promote it to prod. This controlled promotion reduces risk and keeps production stable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

69. How can Terraform be used with Composer for environment setup?

Terraform is an infrastructure-as-code tool, and I can use it to create and manage Composer environments in a repeatable way. With Terraform, I define everything as code:

- Composer environment name, location, machine types, number of nodes.
- Airflow version and image version.
- Network and IAM settings.
- Associated GCS buckets, service accounts, and VPC settings.

When I apply the Terraform config, it creates or updates the Composer environment automatically. This is useful because:

- I can version-control the environment setup in Git.
- I can replicate the same setup for dev, test, and prod with small changes (like environment name or region).
- If the environment is deleted, I can recreate it easily with Terraform.

So Terraform gives me consistency, automation, and easy environment management.

70. How would you manage separate environments for Composer (dev, test, prod)?

I keep separate Composer environments for dev, test, and prod to isolate workloads and reduce risk. Here's how I manage them:

- Create three Composer environments, each with its own GCS bucket, Airflow UI, and configuration.
- Deploy the same DAG code to each environment, but use different Airflow Variables or Connections to point to the correct buckets, datasets, and APIs. This way, the code stays the same, only the config changes.
- Use Git branching or CI/CD to control promotion. For example, commits to dev branch deploy to dev, commits to staging branch deploy to test, and only reviewed/approved commits to main deploy to prod.
- Give broader access to dev (so engineers can test freely), restricted access to test (for QA and validation), and tight control in prod (only release managers or CI/CD pipelines deploy).
- Keep monitoring and alerting tuned differently: dev can be noisier, while prod needs clean alerts for critical pipelines.

This setup ensures experiments don't affect production and lets me promote code step by step with confidence.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

71. How can you orchestrate a daily pipeline that loads GCS data into BigQuery?

I would build a Composer DAG that runs once a day and follows clear, idempotent steps. Typical flow:

- Discover/validate: a small task checks that the expected files for the day exist in GCS (or reads a manifest). Fail-fast if required files are missing.
- Stage: copy or reference the files into a staging area (e.g., gs://bucket/staging/{{ds}}/) and run lightweight schema validation (header, column count, sample rows). If files are Avro/Parquet, this step is mostly checking existence and size.
- Load to BigQuery staging table: use GCSToBigQueryOperator or BigQueryInsertJobOperator to load files into a staging table with WRITE_TRUNCATE for that run. Using a staging table keeps final tables safe.
- Transform / Merge: run a parametrized BigQuery MERGE or INSERT … SELECT that deduplicates and merges staging rows into the production partitioned table (targeting the event_date / ingestion partition). Use MERGE for upserts, WRITE_APPEND for pure appends.
- Post-checks: run validation queries (row counts, null checks, foreign-key-like checks). If checks fail, raise an error and move staging data to a quarantine area.
- Cleanup & notify: clear staging if everything passed and send success/failure notifications. Persist the run watermark (last successful date) in an Airflow Variable or a small control table so incremental logic knows where to resume.

I'd also add retries for idempotency, SLA on the DAG run, and logging of file-level metrics (sizes, row counts) so we can monitor trends and costs.

72. How can Composer coordinate both batch (Dataproc) and streaming (Dataflow) jobs?

I treat Composer as the control plane and use it to deploy, monitor, and coordinate both types of workloads:

- Deploy & manage streaming: use Composer tasks to start or validate a long-running Dataflow streaming job (Flex template or Start operator). Composer can detect if the streaming job is down and restart it, or trigger an alert. I keep the streaming job responsible for low-latency ingestion and writing raw events to a durable store (GCS/BigQuery).
- Orchestrate batch: Composer schedules Dataproc or Dataflow batch jobs to run on a cadence (daily/hourly) that read from the durable store and perform heavy joins, enrichment, and aggregations. Use DataprocSubmitJobOperator or DataflowTemplatedJobStartOperator for batch runs.
- Coordinate interactions: use sensors, manifests, or Pub/Sub signals so batch jobs only run when the streaming layer has produced the expected partitions. For example, a daily DAG checks that the streaming job has written files for ds and then runs the batch transform.
- Handle backfills and late data: expose parameterized backfill DAGs to reprocess specific partitions. If streaming writes late events, Composer can kick off a reprocess for the affected date.
- Monitoring & health: Composer runs health-check DAGs that query Dataflow job metrics (watermark, backlog) and Dataproc job status; SLA misses trigger alerts.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

This separation (streaming = ingestion, batch = completeness) with Composer coordinating gives both low-latency and consistent batch outputs while keeping operations centralized.

73. How can you build data quality checks into a Composer DAG?

I add explicit, automated checks as DAG tasks so quality is part of the pipeline, not an afterthought. Typical checks and structure:

- Schema checks: validate that incoming files match expected schema (columns, types, required fields). Fail-fast and send bad files to quarantine.
- Row-count checks and thresholds: compare expected vs actual record counts (or previous day) and flag large deltas. Use configurable thresholds stored in Variables.
- Uniqueness & integrity checks: run SQL checks in BigQuery for duplicated primary keys, unexpected nulls in required columns, or referential integrity violations.
- Statistical and sanity checks: run checks for outliers, min/max ranges, or distributions that indicate bad data (e.g., >99% null for a field).
- Business rules: verify domain-specific constraints (e.g., financial totals match source, foreign keys resolve).
- Test-run / dry-run: for schema or transformation changes, run checks in a staging dataset before applying to production.
- Failed-check handling: on failure, stop downstream tasks, push the offending data to a dead-letter location, notify owners with diagnostic logs, and create a ticket or message for manual review.
- Observability: persist Q/C metrics (counts, pass/fail) to a monitoring table or send metrics to Cloud Monitoring so dashboards and alerts can be created.

I implement these as separate tasks (e.g., validate_schema >> load_stage >> quality_checks >> merge_to_prod) so failures are visible and recoverable.

74. How would you orchestrate a machine learning workflow (prep → train → deploy) using Composer?

I break the ML lifecycle into clear stages and represent each stage as tasks in a DAG (or as separate DAGs linked together):

- Data preparation: tasks extract raw data, run cleaning and feature engineering (Dataproc/Spark or BigQuery SQL), and write features to a feature store or BigQuery feature tables. Include quality checks and version the dataset (run id, date).
- Train: trigger training jobs (Vertex AI Training, custom training on Dataproc, or Cloud ML). Use templates/operators to start training, pass hyperparameters, and point to the training dataset. Capture model metadata (metrics, artifacts, evaluation results) and push to a model registry or GCS.
- Evaluate & promote: run evaluation tasks to compare new model metrics vs baseline. If metrics meet promotion criteria, mark the model as promotable. If not, fail and notify.
- Deploy: deploy selected model to Vertex AI Endpoint (or to serving infra) via an operator or use a small Cloud Function/CLI step invoked by Composer. Store endpoint details and version metadata.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

- Post-deploy checks: run smoke tests, latency and prediction-quality checks, and monitor online metrics. If regressions occur, Composer can trigger rollback to previous model.
- Automation & traceability: store experiment metadata, hyperparameters, dataset versions, and model artifact locations in a registry or BigQuery audit table so runs are reproducible.
- Separate DAGs & triggers: I often split heavy training into its own DAG (parameterized) and use Composer to coordinate: prep DAG triggers train DAG which triggers deployment DAG on success.

This gives clear gates for quality, repeatability, and safe promotion from experimentation to production serving.

75. How can Composer orchestrate pipelines across multiple GCP projects?

Composer can coordinate cross-project work by using service accounts with appropriate IAM and by referencing resources with fully-qualified names. Practical steps I use:

- Permissions & service accounts: give the Composer environment's service account (or an impersonated service account) IAM roles in the other projects for the resources it needs (BigQuery dataset access, GCS buckets, Dataproc/Dataflow permissions). Prefer least privilege.
- Fully-qualified resource names: in DAG configuration use full identifiers like projects/<project-id>/datasets/<dataset> or gs://<bucket>@<project> so operators target the right project. BigQuery operators accept project and dataset parameters explicitly.
- Separate connections and Variables: store per-project configs (project IDs, buckets, datasets) as Airflow Variables or Connections and reference them in DAGs so the same DAG can run across projects by changing config.
- Network & VPC: ensure networking and firewall rules allow necessary cross-project access (VPC peering or Shared VPC) if Dataproc or other managed components need private connectivity.
- Audit and monitoring: centralize logs to a monitoring project if needed (export logs), or configure Cloud Monitoring in each project but link alerts to a central ops channel.
- CI/CD & deployment: deploy DAGs to the Composer environment in the control project, or maintain DAG copies in each project's Composer environment as appropriate. Use IAM and deployment pipelines to safely promote DAGs across projects.

With these controls, Composer can orchestrate tasks that read from and write to resources in many projects while keeping security, auditing, and operational boundaries clear.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar